

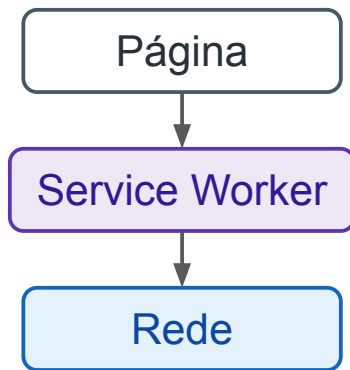
# **Programação Para Dispositivos Móveis 2**

Profº Renato de Souza Garcia

# **CACHE API - ARMAZENANDO RECURSOS NO CACHE**

# Onde paramos?

- Na semana passada construímos:



- Mas ainda temos um problema.

- O Service Worker consegue:

- ✓ ser registrado
- ✓ instalar
- ✓ ativar
- ✓ interceptar requisições

# O problema

- Nosso código:

```
1 self.addEventListener("fetch", event => {  
2     event.respondWith(  
3         fetch(event.request)  
4     );  
5 });
```

- faz:



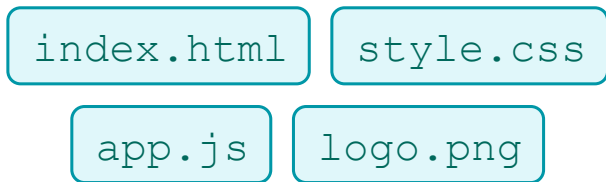
- Se não houver rede:



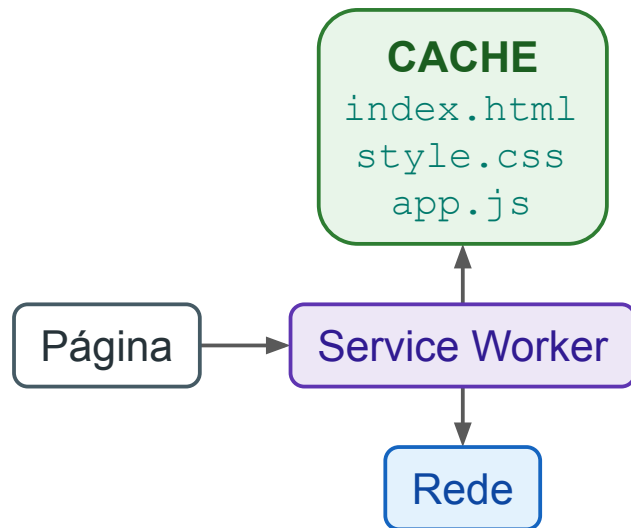
- A aplicação continua dependendo da **Internet**.

# O que precisamos?

- Precisamos armazenar alguns recursos localmente.
- Por exemplo:

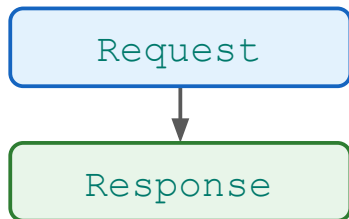


- Assim:

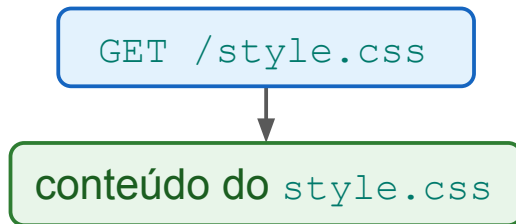


# Cache API

- A **Cache API** permite armazenar pares formados por:



- Exemplo conceitual:



- O navegador fornece `CacheStorage`, que gerencia caches nomeados, e cada `Cache` armazena respostas associadas a requisições.

# Não confunda os caches

- Quando falamos:

```
 caches.open(...)
```

- estamos trabalhando com a:

## Cache Storage API

- Ela é controlada **explicitamente** pelo nosso código.
- Não estamos simplesmente falando do cache HTTP tradicional do navegador.

# caches

- Dentro do Service Worker temos acesso a:

`caches`

- Ele representa o conjunto de caches disponíveis para aquela origem.
- Podemos pensar em:
  - Hoje utilizaremos apenas um:

`caches`

|

├── `agenda-v1`

├── `imagens-v1`

└── `dados-v1`

`agenda-v1`

# Criando ou abrindo um cache

- Utilizamos:

```
 caches.open("agenda-v1")
```

- Exemplo:

```
1 caches.open("agenda-v1")
2   .then(cache => {
3       console.log(
4           "Cache aberto!",
5           cache
6       );
7   });
```

- `caches.open()` retorna uma **Promise** com o objeto `Cache`.
- Se o cache com aquele nome ainda não existir, ele é **criado**.

# Por que dar um nome?

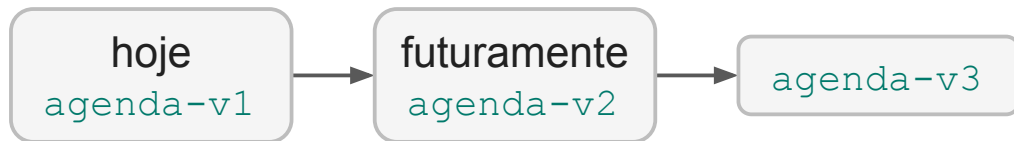
- Exemplo:

```
const CACHE_NAME = "agenda-v1";
```

- Depois:

```
caches.open(CACHE_NAME);
```

- O nome será importante quando nossa aplicação sofrer alterações.



- Isso permite **versionar os recursos armazenados**.

# O que vamos armazenar?

- Vamos definir os recursos essenciais:

```
1 const ARQUIVOS = [  
2     "./",  
3     "./index.html",  
4     "./style.css",  
5     "./app.js"  
6 ];
```

- Podemos incluir também:

```
"./icons/icon-192.png"
```

- e outros arquivos necessários para a interface.

# Pré-cache

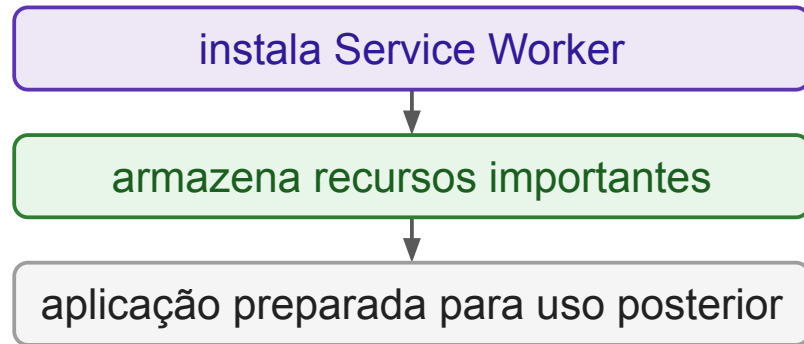
- Queremos armazenar esses arquivos **antes que o usuário precise deles offline.**
- Esse processo é chamado de:

**Precache**

ou:

**Pré-cache**

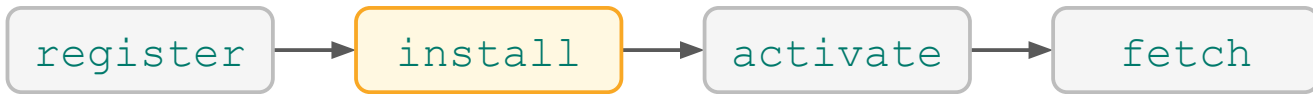
- Ou seja:



- O pré-cache durante o evento `install` é uma técnica comum para tornar recursos essenciais disponíveis offline.

# Qual evento devemos utilizar?

- Na semana anterior vimos:



- O momento ideal para preparar o cache inicial é:

**install**

# Abrindo o cache durante `install`

```
1 const CACHE_NAME = "agenda-v1";  
2  
3 self.addEventListener("install", event => {  
4     caches.open(CACHE_NAME)  
5         .then(cache => {  
6             console.log("Cache criado");  
7         });  
8 });
```

Mas existe um  
detalhe importante.

# Operações assíncronas

- Abrir o cache é uma operação assíncrona:

```
caches.open(CACHE_NAME)
```

- retorna:

```
Promise
```

- O navegador precisa saber:

“O Service Worker ainda está realizando uma tarefa necessária para sua instalação.”

- Para isso utilizamos:

```
event.waitUntil(...)
```

## event.waitUntil()

```
1 self.addEventListener("install", event => {  
2     event.waitUntil(  
3         caches.open(CACHE_NAME)  
4     );  
5 });
```

- `waitUntil()` estende a vida do evento enquanto a Promise estiver pendente.
- No evento `install`, uma rejeição da Promise faz a **instalação falhar**.

# Por que isso importa?

- Sem `event.waitUntil(...)` seria como dizer:

“Comece a armazenar os arquivos...”

- e imediatamente:

“Pronto, terminei a instalação!”

- Com `waitUntil()`:



# Adicionando um arquivo

- Depois de abrir o cache:

```
1 caches.open(CACHE_NAME)
2   .then(cache => {
3     return cache.add("./index.html");
4   });
```

- `cache.add()` realiza a requisição pelo recurso e armazena a resposta resultante no cache.

# E vários arquivos?

- Poderíamos fazer:

```
cache.add("./index.html");  
cache.add("./style.css");  
cache.add("./app.js");
```

- Mas existe:

```
cache.addAll(...)
```

## cache.addAll()

- Primeiro:

```
1 const ARQUIVOS = [  
2     "./",  
3     "./index.html",  
4     "./style.css",  
5     "./app.js"  
6 ];
```

- Depois:

```
cache.addAll(ARQUIVOS);
```

- `addAll()` recebe uma coleção de URLs ou Requests, busca esses recursos e adiciona suas respostas ao cache.

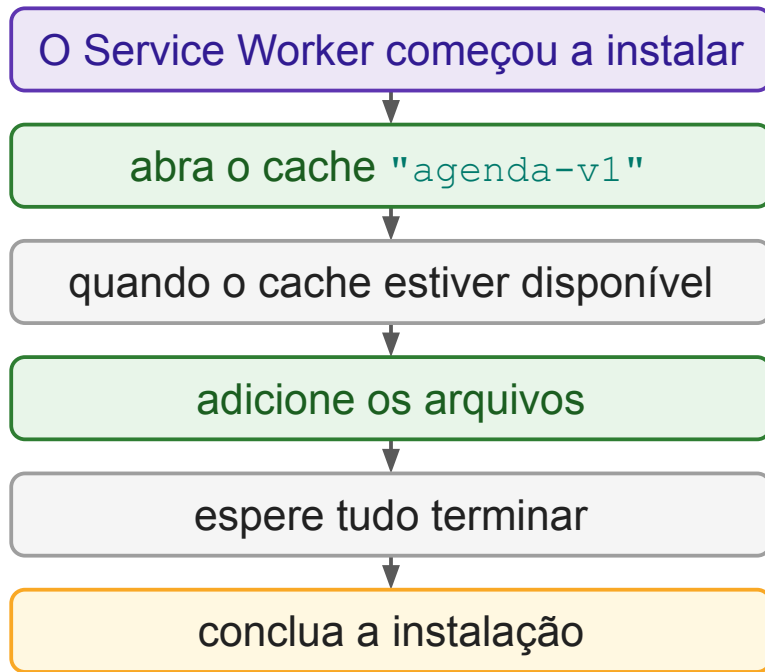
# Nosso `install`

- Agora temos:

```
1 const CACHE_NAME = "agenda-v1";
2
3 const ARQUIVOS = [
4     "./",
5     "./index.html",
6     "./style.css",
7     "./app.js"
8 ];
```

```
9 self.addEventListener("install", event => {
10     event.waitUntil(
11         caches.open(CACHE_NAME)
12             .then(cache => {
13                 return cache.addAll(
14                     ARQUIVOS
15                 );
16             })
17     );
18 });
```

# Leia o código como uma sequência



# Cuidado com `addAll()`

- Considere:

```
1 const ARQUIVOS = [  
2     "./index.html",  
3     "./style.css",  
4     "./nao-existe.js"  
5 ];
```

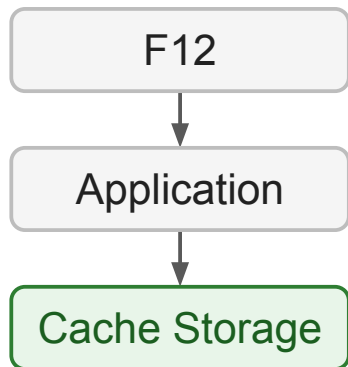
- Se um recurso necessário para `addAll()` não puder ser obtido corretamente, a Promise pode ser rejeitada e o processo de instalação **falhar**.

## Portanto:

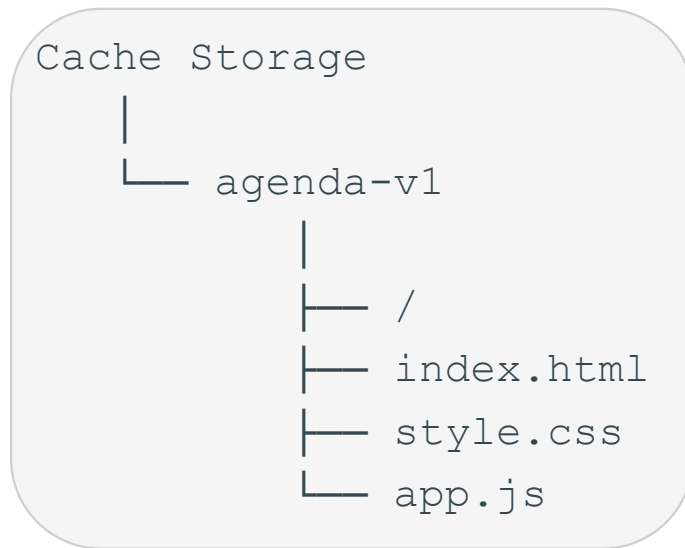
Confira cuidadosamente os caminhos dos arquivos.

# Vamos testar

- Abra:



- Deverá aparecer algo semelhante a:



# Experimento

- Faça uma alteração:

```
const CACHE_NAME = "agenda-v2";
```

- Recarregue a aplicação e atualize o Service Worker.
- Agora observe o DevTools:

agenda-v1

agenda-v2

## Pergunta

Por que agora temos dois caches?

- Ainda não programamos a remoção da versão antiga.
- Faremos isso posteriormente.

**RESPONDENDO PELO CACHE**

# Temos arquivos armazenados

- Nossa arquitetura agora é:

**agenda-v1**

`index.html`  
`style.css`  
`app.js`

- Mas armazenar os arquivos sozinho **não resolve tudo**.
- Precisamos **utilizá-los**.

# Retomando `fetch`

- Na semana passada:

```
1 self.addEventListener("fetch", event => {  
2   console.log(  
3     event.request.url  
4   );  
5 });
```

- Agora, quando uma requisição chegar:

GET /style.css

- queremos perguntar:

Existe uma resposta armazenada para esta requisição?

## `caches.match()`

- Podemos utilizar:

```
caches.match(event.request)
```

- Ele procura uma resposta armazenada correspondente à requisição.
- O resultado é uma **Promise**.
- Quando encontra:
- Quando não encontra:

Response

undefined

# Exemplo

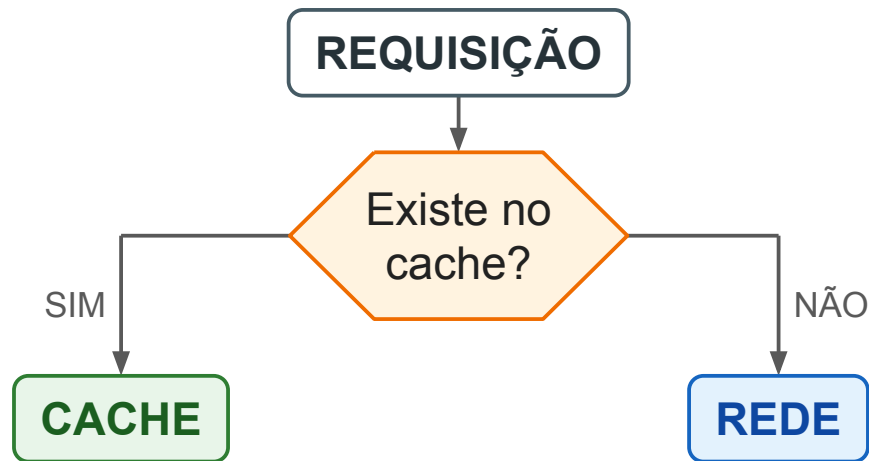
```
1 caches.match(event.request)
2   .then(resposta => {
3       console.log(resposta);
4   });
```

- Podemos testar:

```
1 if (resposta) {
2     console.log(
3         "Encontrado no cache"
4     );
5 }
```

# A decisão

- Queremos:



- Essa é nossa primeira lógica de **resposta offline**.

## respondWith() novamente

- Na semana passada:

```
event.respondWith(  
    fetch(event.request)  
);
```

- Agora:

```
1 event.respondWith(  
2     caches.match(event.request)  
3 );
```

- Mas existe um problema.

# E se não estiver no cache?

- Se:

```
caches.match(event.request)
```

- não encontrar nada,  
precisamos tentar:

```
fetch(event.request)
```

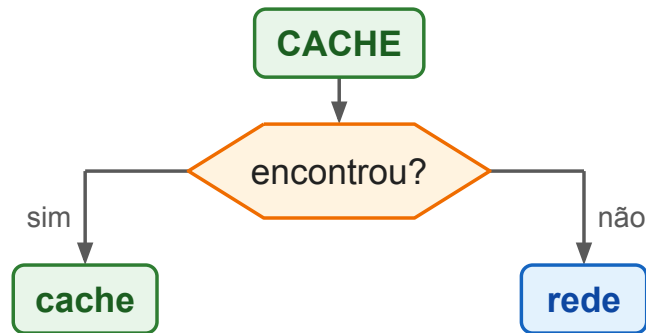
- Então:

```
1 self.addEventListener("fetch", event => {  
2     event.respondWith(  
3         caches.match(event.request)  
4             .then(resposta => {  
5                 if (resposta) {  
6                     return resposta;  
7                 }  
8                 return fetch(  
9                     event.request  
10                );  
11            })  
12    });  
13 });
```

# O mesmo código, mais compacto

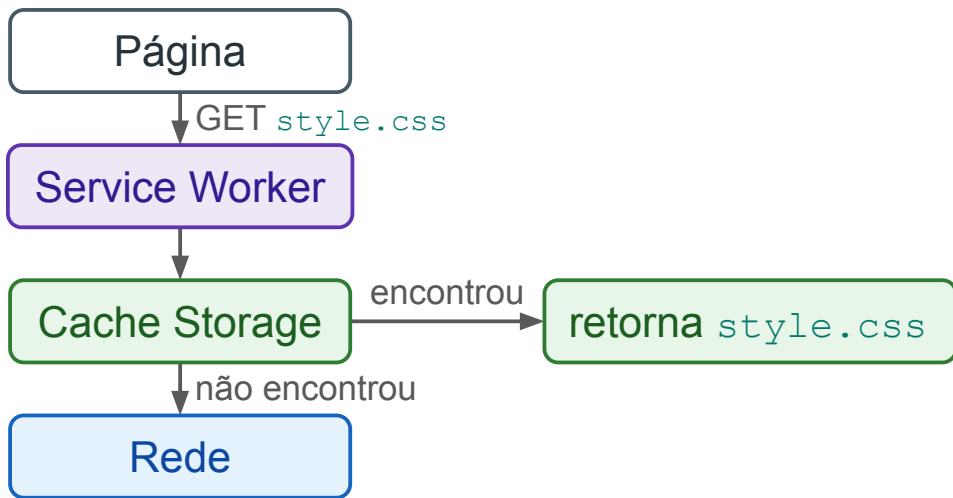
```
1 self.addEventListener("fetch", event => {  
2   event.respondWith(  
3     caches.match(event.request)  
4       .then(resposta => {  
5         return resposta || fetch(event.request);  
6       })  
7   );  
8 });
```

- A lógica continua sendo:



# Fluxo completo

- Quando a página solicita `style.css`, temos:



# Nosso primeiro `sw.js` offline (1/2)

```
1 const CACHE_NAME = "agenda-v1";
2
3 const ARQUIVOS = [
4     "./",
5     "./index.html",
6     "./style.css",
7     "./app.js"
8 ];
```

```
9 self.addEventListener("install", event => {
10     event.waitUntil(
11         caches.open(CACHE_NAME)
12             .then(cache => {
13                 return cache.addAll(ARQUIVOS);
14             })
15     );
16 });
```

## Nosso primeiro `sw.js` offline (2/2)

```
22 self.addEventListener("activate", event => {  
23     console.log(  
24         "Service Worker ativado"  
25     );  
26 });
```

```
27 self.addEventListener("fetch", event => {  
28     event.respondWith(  
29         caches.match(event.request)  
30         .then(resposta => {  
31             return resposta ||  
32                 fetch(event.request);  
33         })  
34     );  
35 });
```

- Esse é o primeiro Service Worker do curso que realmente começa a permitir **funcionamento sem rede**.

# Vamos testar de verdade

## Passo 1

Carregue a aplicação normalmente.

## Passo 2

Confira:

Application



Cache Storage

## Passo 3

DevTools:

Network



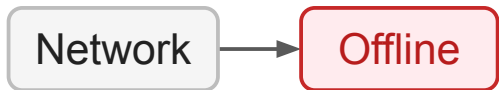
Offline

## Passo 4

Recarregue a página.

# O resultado esperado

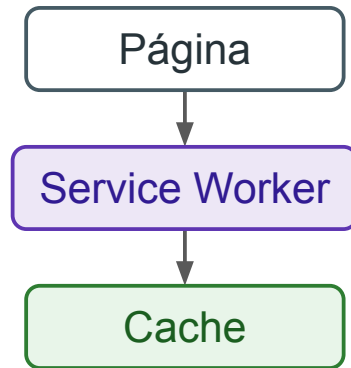
- Mesmo com:



- a aplicação deverá conseguir obter os recursos que foram previamente armazenados:



- porque:



- não depende da rede para esses recursos.

# Atenção!

- Se a página utiliza `logo.png`
- mas nosso array contém apenas:

```
1 const ARQUIVOS = [  
2   "./",  
3   "./index.html",  
4   "./style.css",  
5   "./app.js"  
6 ];
```

- o que acontecerá offline?

✓ `index.html`

✓ `style.css`

✓ `app.js`

✗ `logo.png`

## Por quê?

- O recurso **não** foi previamente armazenado.

# Corrigindo

- Adicione:

```
1 const ARQUIVOS = [  
2     "./",  
3     "./index.html",  
4     "./style.css",  
5     "./app.js",  
6     "./icons/icon-192.png",  
7     "./icons/icon-512.png"  
8 ];
```

- Agora esses recursos também podem ser disponibilizados pelo cache.

# **ATIVIDADE PRÁTICA**

# Desafio

## Faça sua PWA funcionar offline

- Continue a aplicação das Semanas 3 e 4.
- Implemente:

✓ cache nomeado

✓ precache no `install`

✓ `event.waitUntil()`

✓ `cache.addAll()`

✓ interceptação com `fetch`

✓ busca utilizando `caches.match()`

✓ fallback para a rede

# Etapa 1

- Defina:

```
const CACHE_NAME = "minha-pwa-v1";
```

- **Adapte os caminhos ao seu projeto.**

- Depois liste os recursos:

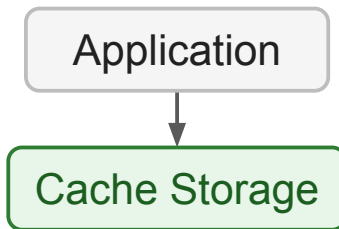
```
1 const ARQUIVOS = [  
2     "./",  
3     "./index.html",  
4     "./style.css",  
5     "./app.js"  
6 ];
```

# Etapa 2

- Implemente:

```
1 self.addEventListener("install", event => {
2     event.waitUntil(
3         caches.open(CACHE_NAME)
4             .then(cache => {
5                 return cache.addAll(
6                     ARQUIVOS
7                 );
8             })
9     );
10 });
```

- Verifique o cache no DevTools.



## Etapa 3

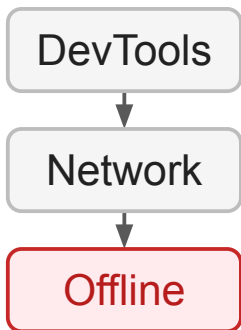
- Implemente:

```
1 self.addEventListener("fetch", event => {  
2     event.respondWith(  
3         caches.match(event.request)  
4             .then(resposta => {  
5                 return resposta ||  
6                     fetch(event.request);  
7             })  
8     );  
9 });
```

- Depois teste com a **rede ativada**.

# Etapa 4

- Agora:



- Recarregue.

## Responda

- 1. A página abriu?
  - 2. O CSS foi aplicado?
  - 3. O JavaScript executou?
  - 4. Todas as imagens apareceram?
  - 5. Alguma requisição falhou?
- Se algo falhou:

**Descubra qual recurso não está armazenado.**

**INVESTIGAÇÃO**

# Encontre o erro

- Considere:

```
1 const ARQUIVOS = [  
2     "./index.html",  
3     "./style.css",  
4     "./ap.js"  
5 ];
```

- Porém o arquivo verdadeiro é `app.js`.
- O que pode acontecer durante `cache.addAll(ARQUIVOS)`?

## Resposta

- A requisição para `ap.js` não terá sucesso.
- A Promise de `addAll()` pode ser rejeitada e a **instalação do novo Service Worker falhar**.

# Outro problema

```
1 self.addEventListener("fetch", event => {  
2     caches.match(event.request)  
3         .then(resposta => {  
4             return resposta ||  
5                 fetch(event.request);  
6         });  
7 });
```

- O que está faltando?

## Resposta

```
event.respondWith(...)
```

- Sem ele, não estamos fornecendo aquela Promise como resposta controlada para o evento `fetch`.

# Corrigindo

```
1 self.addEventListener("fetch", event => {  
2   event.respondWith(  
3     caches.match(event.request)  
4       .then(resposta => {  
5         return resposta || fetch(event.request);  
6       })  
7   );  
8 });  
9
```

- `event.respondWith(...)` entrega a Promise como resposta ao evento `fetch`.

# Questão conceitual

- Considere:

```
const ARQUIVOS = [  
  "./index.html",  
  "./style.css"  
];
```

- O usuário abre a aplicação **offline**.
- O HTML contém:

```
<script src="app.js"></script>
```

## O que provavelmente acontecerá?

- O HTML e CSS podem ser encontrados no cache.
- Mas `app.js` **não foi armazenado**.
- Sem conexão de rede, sua requisição poderá falhar.

# Cache não significa “cachear tudo”

- Um erro comum seria pensar:

“Se cache é bom, vou armazenar tudo.”

- Mas aplicações reais precisam decidir:
  - O que deve ser armazenado?
  - Quando deve ser armazenado?
  - Quando deve ser atualizado?
  - Quando devemos consultar a rede?
  - O que fazer com dados dinâmicos?
- Essas decisões formam as **estratégias de cache**.

# Arquivos estáticos × dados dinâmicos

- Hoje trabalhamos principalmente com:

## Arquivos estáticos

HTML

CSS

JavaScript

ícones

imagens da interface

## Pergunta

Devemos tratar dados de uma API exatamente como tratamos `style.css`?

- Mas imagine:

```
fetch(  
  "https://api.exemplo.com/tarefas"  
)
```

- As tarefas podem mudar constantemente.

**Não necessariamente.**

# Um problema novo

- Imagine:

## Segunda-feira:

- cacheamos as tarefas.

## Terça-feira:

- o professor adiciona uma tarefa nova.

- Então precisamos equilibrar:

VELOCIDADE

+

OFFLINE

+

ATUALIZAÇÃO

- Se sempre respondermos primeiro com:

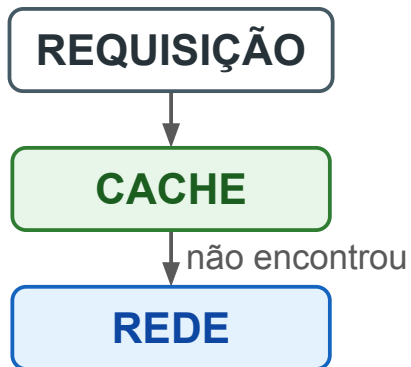
CACHE

- o estudante pode visualizar:

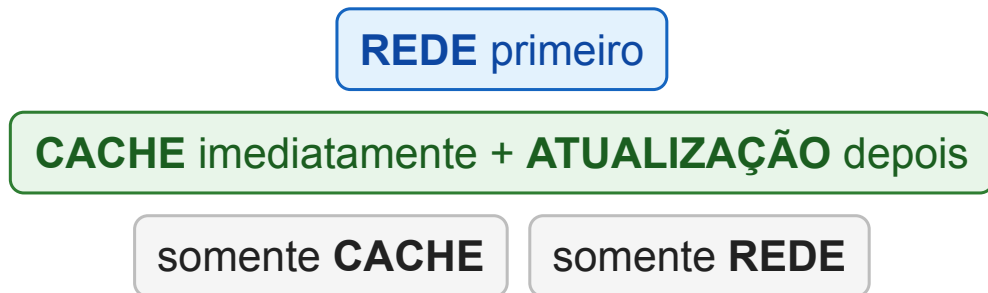
DADOS ANTIGOS

# Esse é o próximo passo

- Até agora:



- Mas outras possibilidades existem:



- Não aprofundaremos essas estratégias hoje.

**FECHAMENTO**

# O que aprendemos?

## Cache Storage

```
caches.open(...)
```

Abre ou cria um cache.

## Adicionar vários recursos

```
cache.addAll(...)
```

Busca e armazena recursos.

## Procurar uma resposta

```
caches.match(...)
```

Procura uma resposta armazenada.

## Manter a instalação ativa

```
event.waitUntil(...)
```

Aguarda operações assíncronas importantes.

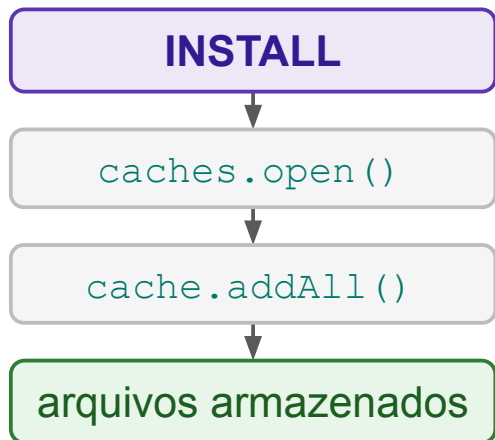
## Responder a uma requisição

```
event.respondWith(...)
```

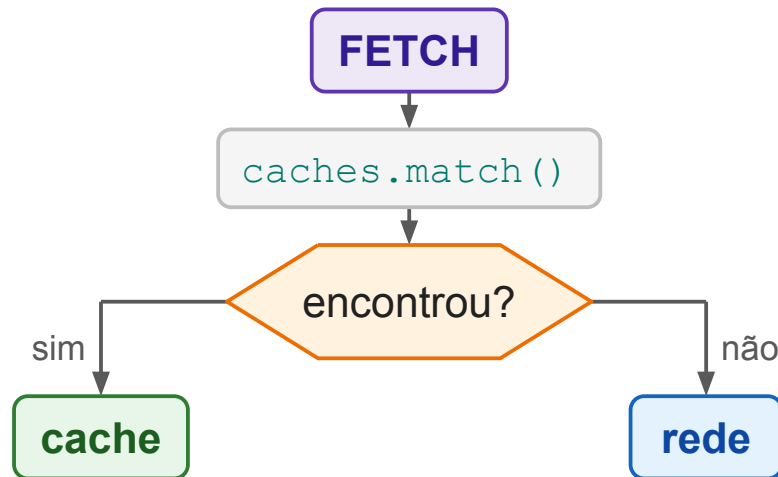
Define a resposta utilizada pelo evento `fetch`.

# Fluxo aprendido

## Durante a instalação



## Durante a utilização



# Evolução da nossa PWA

## Semana 3

**Manifest**



identidade  
+  
instalação

## Semana 4

**Service Worker**



interceptação

## Semana 5

**Cache API**



armazenamento



resposta sem rede

# Agora temos uma PWA offline?

Para parte da aplicação:

**Sim.**

- Os recursos armazenados podem ser utilizados sem rede.

- Mas ainda temos perguntas:
  - E APIs?
  - E dados atualizados?
  - E páginas não armazenadas?
  - E alterações feitas pelo usuário?
  - E conexão que cai durante o uso?

É aí que entra o conceito de:

**Offline First**

# Próxima semana

## Offline First

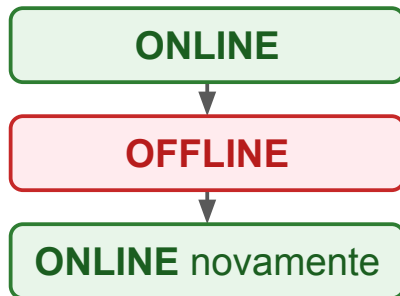
- A pergunta deixa de ser:

“Como fazer um site funcionar sem Internet?”

- E passa a ser:

**“Como projetar a aplicação supondo que a conexão pode estar lenta, instável ou indisponível?”**

- Vamos trabalhar situações como:



- e começar a pensar na aplicação como um sistema que precisa ser **resiliente à conectividade**.

# Desafio final

- Antes da próxima aula, observe aplicativos que você utiliza diariamente.
- Pense:

**O que ainda funciona quando sua Internet cai?**

- Considere:

WhatsApp

Google Maps

Spotify

YouTube

aplicativos bancários

aplicativos de transporte

## Pergunta

- Todos deveriam possuir exatamente o mesmo comportamento offline?

**Não.**

- A resposta depende dos dados e da finalidade da aplicação.

**CÓDIGO FINAL**

## app.js

```
1 if ("serviceWorker" in navigator) {  
2     navigator.serviceWorker  
3         .register("./sw.js")  
4         .then(() => {  
5             console.log("Service Worker registrado");  
6         })  
7         .catch(erro => {  
8             console.error("Erro no Service Worker:", erro);  
9         });  
10 }
```

## sw.js (1/2)

```
1 const CACHE_NAME = "agenda-v1";
2
3 const ARQUIVOS = [
4     "./",
5     "./index.html",
6     "./style.css",
7     "./app.js",
8     "./icons/icon-192.png",
9     "./icons/icon-512.png"
10 ];
```

```
11 self.addEventListener("install", event => {
12     console.log("Instalando Service Worker...");
13     event.waitUntil(
14         caches.open(CACHE_NAME)
15             .then(cache => {
16                 console.log("Armazenando arquivos...");
17                 return cache.addAll(ARQUIVOS);
18             })
19     );
20 });
```

## sw.js (2/2)

```
35 self.addEventListener("activate", event => {
36     console.log(
37         "Service Worker ativado"
38     );
39 });
40
41 self.addEventListener("fetch", event => {
42     event.respondWith(
43         caches.match(event.request)
44             .then(resposta => {
45                 if (resposta) {
46                     console.log(
47                         "Cache:",
48                         event.request.url
49                     );
50                     return resposta;
51                 }
```

```
52         console.log(
53             "Rede:",
54             event.request.url
55         );
56         return fetch(
57             event.request
58         );
59     })
60 );
61 });
```

Fim